

AWS Deployment — Pure CLI

Empty AWS account to a running, secured, monitored system. **No scripts.** Every step is a command you type, so you can see exactly what is created and stop at any line.

Companion to [AWS-DEPLOYMENT-COMPLETE.md](#), which runs `scripts/01..05` instead. Same result. Use this one when you want to understand each resource, or when a script has failed and you need to take over by hand.

Assumes the application code is done and tests pass. This is deployment only.

How to read this

Every block is copy-pasteable. Variables set in one step are used in later ones, so **keep one terminal open for the whole run**.

Where something is genuinely easy to get wrong, there is a  note. Each of those is a mistake that actually happened, not a hypothetical.

0 — Prerequisites

```
aws --version          # v2.x
kubect1 version --client
eksctl version
helm version
docker --version
jq --version
```

```
aws configure          # access key, secret, region, json
aws sts get-caller-identity # must return your account
```

Set these once. Everything below uses them.

```
export AWS_REGION=ap-south-1
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export CLUSTER=ecommerce-cluster
export DOMAIN=yourdomain.com          # one you actually own
export PROJECT=$HOME/Desktop/ecommerce
cd $PROJECT
```

1 — VPC and subnets

Three tiers across two availability zones. Public holds the load balancer, private holds the nodes, data is reserved for managed datastores.

```
VPC_ID=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
--query 'Vpc.VpcId' --output text)
```

```
aws ec2 create-tags --resources $VPC_ID --tags Key=Name,Value=ecommerce-vpc
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-hostnames
aws ec2 modify-vpc-attribute --vpc-id $VPC_ID --enable-dns-support
echo "VPC: $VPC_ID"
```

DNS hostnames are not optional — EKS needs them for private endpoint resolution.

```
IGW_ID=$(aws ec2 create-internet-gateway \
--query 'InternetGateway.InternetGatewayId' --output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID
```

```
PUB1=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block 10.0.1.0/24 \
--availability-zone ${AWS_REGION}a --query 'Subnet.SubnetId' --output text)
PUB2=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block 10.0.2.0/24 \
--availability-zone ${AWS_REGION}b --query 'Subnet.SubnetId' --output text)
PRIV1=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block 10.0.3.0/24 \
--availability-zone ${AWS_REGION}a --query 'Subnet.SubnetId' --output text)
PRIV2=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block 10.0.4.0/24 \
--availability-zone ${AWS_REGION}b --query 'Subnet.SubnetId' --output text)
```

```
aws ec2 create-tags --resources $PUB1 --tags Key=Name,Value=public-az1
aws ec2 create-tags --resources $PUB2 --tags Key=Name,Value=public-az2
aws ec2 create-tags --resources $PRIV1 --tags Key=Name,Value=private-az1
aws ec2 create-tags --resources $PRIV2 --tags Key=Name,Value=private-az2
```

1.1 Subnet discovery tags — do not skip these

The AWS Load Balancer Controller finds subnets by tag. Without them it cannot place a load balancer and fails with a message about subnet discovery that reads like a permissions problem.

```
for s in $PUB1 $PUB2; do
  aws ec2 create-tags --resources $s \
    --tags Key=kubernetes.io/role/elb,Value=1 \
      Key=kubernetes.io/cluster/$CLUSTER,Value=shared
done

for s in $PRIV1 $PRIV2; do
  aws ec2 create-tags --resources $s \
    --tags Key=kubernetes.io/role/internal-elb,Value=1 \
      Key=kubernetes.io/cluster/$CLUSTER,Value=shared
done
```

2 — Routing

Public subnets route to the internet gateway.

```
RT_PUB=$(aws ec2 create-route-table --vpc-id $VPC_ID \
  --query 'RouteTable.RouteTableId' --output text)
aws ec2 create-route --route-table-id $RT_PUB \
  --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW_ID
aws ec2 associate-route-table --route-table-id $RT_PUB --subnet-id $PUB1
aws ec2 associate-route-table --route-table-id $RT_PUB --subnet-id $PUB2
```

Private subnets route outbound through a NAT gateway, which lives in a public subnet and needs an Elastic IP.

```
EIP=$(aws ec2 allocate-address --domain vpc --query 'AllocationId' --output text)
NAT=$(aws ec2 create-nat-gateway --subnet-id $PUB1 --allocation-id $EIP \
  --query 'NatGateway.NatGatewayId' --output text)

aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT # ~2 minutes
```

```
RT_PRIV=$(aws ec2 create-route-table --vpc-id $VPC_ID \
  --query 'RouteTable.RouteTableId' --output text)
aws ec2 create-route --route-table-id $RT_PRIV \
  --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT
aws ec2 associate-route-table --route-table-id $RT_PRIV --subnet-id $PRIV1
aws ec2 associate-route-table --route-table-id $RT_PRIV --subnet-id $PRIV2
```

⚠ **One NAT gateway is a single point of failure.** Both private subnets egress through one AZ. If that AZ fails, the whole private tier loses outbound internet — image pulls, SES, every external API. Production wants one NAT per AZ. It roughly doubles the NAT bill, and that is the trade being made.

3 — Security groups

```
ALB_SG=$(aws ec2 create-security-group --group-name alb-sg \
  --description "ALB" --vpc-id $VPC_ID --query 'GroupId' --output text)
aws ec2 authorize-security-group-ingress --group-id $ALB_SG \
  --protocol tcp --port 80 --cidr 0.0.0.0/0
aws ec2 authorize-security-group-ingress --group-id $ALB_SG \
  --protocol tcp --port 443 --cidr 0.0.0.0/0

EKS_SG=$(aws ec2 create-security-group --group-name eks-sg \
  --description "EKS nodes" --vpc-id $VPC_ID --query 'GroupId' --output text)
# only the ALB may reach the gateway port
aws ec2 authorize-security-group-ingress --group-id $EKS_SG \
  --protocol tcp --port 8000 --source-group $ALB_SG
# nodes talk to each other on ephemeral ports
aws ec2 authorize-security-group-ingress --group-id $EKS_SG \
  --protocol tcp --port 1024-65535 --source-group $EKS_SG
```

⚠ **Security groups do not separate your pods from your databases here.** They filter per-ENI, so they govern node and load-balancer traffic. Pod-to-pod stays on the CNI overlay and never passes through them. If your datastores run as StatefulSets — as they do in this repo — the thing that keeps `order-service` away from `postgres-user` is the NetworkPolicy in §12, not a `db-sg`. Creating a `db-sg` and believing it protects anything is worse than not creating one.

4 — NACLs and flow logs

```
NACL=$(aws ec2 create-network-acl --vpc-id $VPC_ID \
  --query 'NetworkAcl.NetworkAclId' --output text)

aws ec2 create-network-acl-entry --network-acl-id $NACL --ingress \
  --rule-number 100 --protocol tcp --port-range From=443,To=443 \
  --cidr-block 0.0.0.0/0 --rule-action allow
aws ec2 create-network-acl-entry --network-acl-id $NACL --ingress \
  --rule-number 110 --protocol tcp --port-range From=80,To=80 \
  --cidr-block 0.0.0.0/0 --rule-action allow
# return traffic – NACLs are stateless, unlike security groups
aws ec2 create-network-acl-entry --network-acl-id $NACL --ingress \
  --rule-number 120 --protocol tcp --port-range From=1024,To=65535 \
  --cidr-block 0.0.0.0/0 --rule-action allow
aws ec2 create-network-acl-entry --network-acl-id $NACL --egress \
  --rule-number 100 --protocol -1 --port-range From=0,To=65535 \
  --cidr-block 0.0.0.0/0 --rule-action allow
```

4.1 Attaching it — the part that catches everyone

`--association-id` wants the id of the association between a subnet and the NACL it currently uses (`aclassoc-...`). **It is not the subnet id**. Passing a subnet id fails with `InvalidAssociationID.NotFound` , and under `set -e` in a script everything after that line is skipped — leaving a NACL that exists and protects nothing.

```
for SUBNET in $PUB1 $PUB2; do
  ASSOC=$(aws ec2 describe-network-acls \
    --filters "Name=association.subnet-id,Values=$SUBNET" \
    --query "NetworkAcls[.Associations[?SubnetId=='$SUBNET'].NetworkAclAssociationId" \
    --output text)
  aws ec2 replace-network-acl-association \
    --network-acl-id $NACL --association-id $ASSOC
  echo "$SUBNET -> $NACL"
done
```

Flow logs, so rejected traffic is visible after the fact:

```
FL_BUCKET=ecommerce-flowlogs-$ACCOUNT_ID
aws s3 mb s3://$FL_BUCKET --region $AWS_REGION
aws ec2 create-flow-logs --resource-type VPC --resource-ids $VPC_ID \
  --traffic-type ALL --log-destination-type s3 \
  --log-destination arn:aws:s3:::$FL_BUCKET
```

5 — The EKS cluster

`eksctl` is used here rather than raw `aws eks create-cluster` , because the raw call still leaves you to build the node role, the launch template, the `aws-auth` entry and the OIDC provider by hand. This is the one place a helper earns its keep.

```
cat > /tmp/cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: $CLUSTER
  region: $AWS_REGION
  version: "1.29"
vpc:
  id: $VPC_ID
  subnets:
    public:
      ${AWS_REGION}a: { id: $PUB1 }
      ${AWS_REGION}b: { id: $PUB2 }
    private:
      ${AWS_REGION}a: { id: $PRIV1 }
      ${AWS_REGION}b: { id: $PRIV2 }
iam:
  withOIDC: true
managedNodeGroups:
  - name: general
    instanceType: t3.large
    desiredCapacity: 3
    minSize: 2
    maxSize: 6
    privateNetworking: true
    volumeSize: 30
addons:
  - name: vpc-cni
    configurationValues: '{"enableNetworkPolicy":"true"}'
  - name: coredns
  - name: kube-proxy
  - name: aws-ebs-csi-driver
  wellKnownPolicies:
    ebsCSIDriver: true
EOF
```

```
eksctl create cluster -f /tmp/cluster.yaml # 15-20 minutes
kubectl get nodes
```

Two lines in that file matter more than the rest:

- `enableNetworkPolicy: "true"` — without it the VPC CNI ignores every NetworkPolicy you apply. They will show up in `kubectl get netpol` and block nothing at all.
- `aws-ebs-csi-driver` — without it a `PersistentVolumeClaim` has nothing to bind to and every StatefulSet sits `Pending` forever.

5.1 A default StorageClass

```
cat <<EOF | kubectl apply -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  type: gp3
  encrypted: "true"
EOF
```

```
kubectl get storageclass
```

`WaitForFirstConsumer` matters: without it a volume can be created in an AZ where the pod cannot be scheduled, and the pod never starts.

6 — Cluster add-ons

6.1 AWS Load Balancer Controller

It needs its own IAM policy and an IRSA-bound service account. Static keys in a Secret would work and are the wrong answer — IRSA gives the pod short-lived credentials with no secret to leak.

```
curl -s0 https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.7.2/docs/install/iam_policy.json
```

```
POLICY_ARN=$(aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json \
  --query 'Policy.Arn' --output text 2>/dev/null \
  || aws iam list-policies --scope Local \
    --query "Policies[?PolicyName=='AWSLoadBalancerControllerIAMPolicy'].Arn" \
  --output text)
```

```
eksctl create iamserviceaccount \
  --cluster=$CLUSTER --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --attach-policy-arn=$POLICY_ARN \
  --approve
```

```
helm repo add eks https://aws.github.io/eks-charts && helm repo update
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=$CLUSTER \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=$AWS_REGION \
  --set vpcId=$VPC_ID
```

```
kubectl -n kube-system rollout status deploy/aws-load-balancer-controller
```

6.2 metrics-server

EKS does not ship it, and every HPA depends on it. Without it an HPA reports `<unknown>` targets and never scales.

```
kubect1 apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubect1 -n kube-system rollout status deploy/metrics-server
```


6.3 Sealed Secrets controller

```
kubectl apply -f https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.27.1/controller.yaml
kubectl -n kube-system rollout status deploy/sealed-secrets-controller
```

7 — Images into ECR

```
SERVICES="user product order inventory payment notification"
REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

for s in $SERVICES; do
  aws ecr describe-repositories --repository-names ecommerce/$s-service >/dev/null 2>&1 \
  || aws ecr create-repository --repository-name ecommerce/$s-service \
    --image-scanning-configuration scanOnPush=true \
    --encryption-configuration encryptionType=AES256
done

aws ecr get-login-password --region $AWS_REGION \
| docker login --username AWS --password-stdin $REGISTRY
```

Tag from the commit — but only if the tree is clean, or the tag lies:

```
IMAGE_TAG=$(git rev-parse --short HEAD)
[ -n "$(git status --porcelain)" ] && IMAGE_TAG="$IMAGE_TAG-dirty-$(date +%s)"
echo "tag: $IMAGE_TAG"
```

△ With `imagePullPolicy: IfNotPresent`, reusing a tag for a different build means a node that already cached the old image keeps running the **old code** while every dashboard reports the new tag. The `-dirty` suffix makes that impossible.

```
mvn -B -T 1C clean package -DskipTests

for s in $SERVICES; do
  docker build -t $REGISTRY/ecommerce/$s-service:$IMAGE_TAG ./s-service
  docker push $REGISTRY/ecommerce/$s-service:$IMAGE_TAG
done
```

8 — Certificate and DNS

```
CERT_ARN=$(aws acm request-certificate \
  --domain-name $DOMAIN \
  --subject-alternative-names ".*$DOMAIN" \
  --validation-method DNS \
  --query 'CertificateArn' --output text)

aws acm describe-certificate --certificate-arn $CERT_ARN \
  --query 'Certificate.DomainValidationOptions[0].ResourceRecord'
```

Create that CNAME at your DNS provider, then wait:

```
aws acm wait certificate-validated --certificate-arn $CERT_ARN
```

△ ACM will never validate a `.local` domain, or any domain you cannot create a public DNS record for. If the wait hangs forever, that is why.

9 — ALB, target group, listeners

```
ALB_ARN=$(aws elbv2 create-load-balancer \
  --name ecommerce-alb \
  --subnets $PUB1 $PUB2 \
  --security-groups $ALB_SG \
  --scheme internet-facing --type application --ip-address-type ipv4 \
  --query 'LoadBalancers[0].LoadBalancerArn' --output text)

ALB_DNS=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB_ARN \
```

```
--query 'LoadBalancers[0].DNSName' --output text)
echo "ALB: $ALB_DNS"
```

The target group points at **pod IPs**, and health-checks Kong's status port — not the proxy port, and not the admin API.

```
TG_ARN=$(aws elbv2 create-target-group \
--name ecommerce-kong-tg \
--protocol HTTP --port 8000 \
--vpc-id $VPC_ID --target-type ip \
--health-check-protocol HTTP --health-check-port 8100 \
--health-check-path /status \
--health-check-interval-seconds 15 --health-check-timeout-seconds 5 \
--healthy-threshold-count 2 --unhealthy-threshold-count 3 \
--matcher HttpCode=200 \
--query 'TargetGroups[0].TargetGroupArn' --output text)
```

```
aws elbv2 modify-target-group-attributes --target-group-arn $TG_ARN \
--attributes Key=deregistration_delay.timeout_seconds,Value=30
```

Listeners: 443 serves, 80 redirects.

```
aws elbv2 create-listener --load-balancer-arn $ALB_ARN \
--protocol HTTPS --port 443 \
--ssl-policy ELBSecurityPolicy-TLS13-1-2-2021-06 \
--certificates CertificateArn=$CERT_ARN \
--default-actions Type=forward,TargetGroupArn=$TG_ARN
```

```
aws elbv2 create-listener --load-balancer-arn $ALB_ARN \
--protocol HTTP --port 80 \
--default-actions '[{"Type": "redirect", "RedirectConfig": {"Protocol": "HTTPS", "Port": "443", "StatusCode": "HTTP_301"}}]'
```

Point DNS at it:

```
ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name $DOMAIN \
--query 'HostedZones[0].Id' --output text | cut -d/ -f3)
ALB_ZONE=$(aws elbv2 describe-load-balancers --load-balancer-arns $ALB_ARN \
--query 'LoadBalancers[0].CanonicalHostedZoneId' --output text)
```

```
aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID \
--change-batch "{
  \"Changes\": [{
    \"Action\": \"UPSERT\",
    \"ResourceRecordSet\": {
      \"Name\": \"api.$DOMAIN\",
      \"Type\": \"A\",
      \"AliasTarget\": {
        \"HostedZoneId\": \"$ALB_ZONE\",
        \"DNSName\": \"$ALB_DNS\",
        \"EvaluateTargetHealth\": true
      }
    }
  ]
}"
```

10 — Secrets, sealed

A Kubernetes `Secret` is base64 — encoding, not encryption. Never commit one with real values.

```
kubect1 create namespace ecommerce
kubect1 create namespace monitoring
```

```
# fill in real values first
cp k8s/services/secrets.example.yaml /tmp/secrets.yaml
$EDITOR /tmp/secrets.yaml
```

```
kubeseal --controller-namespace kube-system \
--controller-name sealed-secrets-controller \
--format yaml < /tmp/secrets.yaml > k8s/security/sealed-secrets.yaml
```

```
kubectl apply -f k8s/security/sealed-secrets.yaml
kubectl -n ecommerce get secret app-secrets # controller creates this
rm /tmp/secrets.yaml
```

⚠ **The sealing key belongs to this cluster.** The `sealed-secrets.yaml` in the repository was sealed against a different one and cannot be decrypted here. Seal your own. After any cluster rebuild, seal again — the symptom otherwise is pods that never get their `app-secrets`.

11 — Deploy the application

```
kubectl apply -f k8s/services/configmap.yaml
kubectl apply -f k8s/postgres/postgres.yaml
kubectl apply -f k8s/redis/redis.yaml
kubectl apply -f k8s/kafka/kafka.yaml

kubectl -n ecommerce wait --for=condition=ready pod -l app=postgres-user --timeout=300s
kubectl -n ecommerce wait --for=condition=ready pod -l app=redis --timeout=180s
kubectl -n ecommerce wait --for=condition=ready pod -l app=kafka --timeout=300s
kubectl -n ecommerce get pvc # every one must be Bound
```

If a PVC is `Pending`, the EBS CSI driver or the StorageClass from \$5.1 is missing. Nothing downstream will work until that is fixed.

```
kubectl apply -f k8s/kong/kong.yaml
kubectl -n ecommerce rollout status deploy/kong

sed "s|REGISTRY|$REGISTRY|g; s|IMAGE_TAG|$IMAGE_TAG|g" k8s/services/deployments.yaml \
| kubectl apply -f -

for s in user product order inventory payment notification; do
  kubectl -n ecommerce rollout status deploy/$s-service --timeout=300s
done
```

12 — Attach Kong to the ALB

The controller keeps the target group in step with the pods. Registering IPs by hand works exactly until the first restart, after which the ALB is holding a dead address.

```
sed "s|TG_ARN_PLACEHOLDER|$TG_ARN|; s|ALB_SG_PLACEHOLDER|$ALB_SG|" \
k8s/alb/targetgroupbinding.yaml | kubectl apply -f -

sleep 30
aws elbv2 describe-target-health --target-group-arn $TG_ARN \
--query 'TargetHealthDescriptions[].[IP:Target.Id,State:TargetHealth.State]' --output table
kubectl -n ecommerce get pods -l app=kong -o jsonpath='{range .items[*]}{.status.podIP}{"\n"}{end}'
```

The two lists must match.

⚠ `serviceRef.port` in that file is the **Service** port (80), not the container port (8000). Give it the container port and the controller reports `Successfully reconciled` while logging `BackendNotFound`, and the target group stays empty. Always check the targets, not the status.

13 — Lock the network down

```
kubectl apply -f k8s/security/networkpolicy.yaml
kubectl -n ecommerce get netpol # 17

Verify, because a policy that applies is not a policy that blocks:

kubectl -n ecommerce run np-test --image=busybox:1.36 --restart=Never \
--command -- sleep 300

# all four must fail
kubectl -n ecommerce exec np-test -- timeout 4 nc -z postgres-user 5432
kubectl -n ecommerce exec np-test -- timeout 4 nc -z redis 6379
kubectl -n ecommerce exec np-test -- timeout 4 nc -z kafka 9092
kubectl -n ecommerce exec np-test -- timeout 4 nc -z 1.1.1.1 443

kubectl -n ecommerce delete pod np-test
```

If any of those succeeds, the VPC CNI network policy agent is off — see §5.

14 — Monitoring

```
kubectl apply -f k8s/monitoring/  
kubectl -n monitoring rollout status deploy/prometheus  
kubectl -n monitoring rollout status deploy/grafana
```

The whole directory: Prometheus mounts `prometheus-rules` and Grafana mounts `grafana-dashboards`, both defined in files other than `monitoring.yaml`. Applying only that one leaves both pods in `ContainerCreating`.

14.1 The checkpoint

```
kubectl -n monitoring port-forward svc/prometheus 9090:9090 &
curl -s 'http://localhost:9090/api/v1/targets?state=active' \
| jq -r '.data.activeTargets[] | "\(.health) \(.labels.job)'"
```

Every line must say up . Deploying Prometheus is not monitoring. This stack once ran with all ten targets down for the life of a cluster, and nobody noticed because Grafana was 1/1 Running and answering "database": "ok" throughout — it stores nothing and simply drew empty panels.

```
for q in orders_pending_oldest_age_seconds orders_created_total kafka_consumergroup_lag; do
  echo -n "$q -> "
  curl -s "http://localhost:9090/api/v1/query?query=$q" | jq '.data.result | length'
done
```

15 — Verify

```
API=https://api.$DOMAIN
```

```
curl -s $API/api/products # 200, public catalogue
bash .github/smoke-test.sh # 29/29, run against KONG_URL=$API
```

The denial checks are the point. Each of these returned **200** before the services verified their own tokens:

```
BODY='{ "shippingAddress": "x", "items": [{ "productId": 1, "productName": "p", "quantity": 1, "price": 10 } ] }'
curl -s -o /dev/null -w "orders no token -> %{http_code}\n" -X POST $API/api/orders -H 'Content-Type: application/json' -d "$BODY"
curl -s -o /dev/null -w "restock no token -> %{http_code}\n" -X POST $API/api/inventory/1/restock -H 'Content-Type: application/json' -d '{"
```

Both must be 403 .

16 — Teardown, in dependency order

Order matters. AWS refuses to delete a VPC that still has anything in it, and the error does not tell you what.

```
kubectl delete -f k8s/alb/targetgroupbinding.yaml --ignore-not-found
eksctl delete cluster --name $CLUSTER --wait

aws elbv2 delete-listener --listener-arn $(aws elbv2 describe-listeners \
--load-balancer-arn $ALB_ARN --query 'Listeners[0].ListenerArn' --output text)
aws elbv2 delete-load-balancer --load-balancer-arn $ALB_ARN
sleep 30
aws elbv2 delete-target-group --target-group-arn $TG_ARN

# NAT first, then the address it holds
aws ec2 delete-nat-gateway --nat-gateway-id $NAT
aws ec2 wait nat-gateway-deleted --nat-gateway-ids $NAT
aws ec2 release-address --allocation-id $EIP

aws ec2 detach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID
aws ec2 delete-internet-gateway --internet-gateway-id $IGW_ID
for s in $PUB1 $PUB2 $PRIV1 $PRIV2; do aws ec2 delete-subnet --subnet-id $s; done
aws ec2 delete-vpc --vpc-id $VPC_ID
```

⚠ **The Elastic IP is the one people forget.** Deleting the NAT gateway does not release it, and an unattached EIP is billed by the hour, quietly, forever. Check before you walk away:

```
aws ec2 describe-addresses --query 'Addresses[?AssociationId==`null`].[PublicIp,AllocationId]' --output table
aws elbv2 describe-load-balancers --query 'LoadBalancers[].LoadBalancerName'
aws ec2 describe-nat-gateways --filter Name=state,Values=available --query 'NatGateways[].NatGatewayId'
```

All three should come back empty.

Where things go wrong

Every one of these happened. None is hypothetical.

Symptom	Cause
<code>InvalidAssociationID.NotFound</code>	subnet id passed where an <code>aclassoc-</code> id belongs (§4.1)
Every <code>StatefulSet</code> <code>Pending</code>	no EBS CSI driver or no default <code>StorageClass</code> (§5)
NetworkPolicies applied, nothing blocked	VPC CNI network policy agent not enabled (§5)
Target group empty, controller says "Successfully reconciled"	<code>serviceRef.port</code> is the container port, not the Service port (§12)
ALB serving 503 after a deploy	targets registered by hand instead of by <code>TargetGroupBinding</code> (§12)
Pods never receive <code>app-secrets</code>	<code>SealedSecret</code> sealed against a different cluster (§10)
Grafana healthy, every panel empty	Prometheus scraping nothing — check target health, not Grafana (§14.1)
Grafana panels say "No data" with an error badge	<code>datasource uid</code> not declared, so the provisioned dashboard names one that does not exist
HPA stuck at <code><unknown></code>	<code>metrics-server</code> not installed (§6.2)
New tag deployed, old behaviour	a reused tag plus <code>imagePullPolicy: IfNotPresent</code> (§7)
Bill keeps running after teardown	unreleased Elastic IP (§16)